Reliability Engineering and System Safety 106 (2012) 45–60

Contents lists available at SciVerse ScienceDirect

Reliability Engineering and System Safety

journal homepage: www.elsevier.com/locate/ress

Multi-objective optimization of design and testing of safety instrumented systems with MooN voting architectures using a genetic algorithm

A.C. Torres-Echeverría [a,*], S. Martorell [b], H.A. Thompson [a]

[a] Department of Automatic Control and System Engineering, The University of Sheffield, Mappin Street, Sheffield S1 3JD, UK
[b] Department of Chemical and Nuclear Engineering, Polytechnic University of Valencia, Camí de Vera sn, 4602 Valencia, Spain

ARTICLE INFO

Article history:
Received 1 July 2011
Received in revised form
24 February 2012
Accepted 8 March 2012
Available online 29 March 2012

Keywords:
Safety system
Optimization
Genetic algorithms
k-out-of-*n*
Probability of failure on demand
Spurious trip rate

ABSTRACT

This paper presents the optimization of design and test policies of safety instrumented systems using MooN voting redundancies by a multi-objective genetic algorithm. The objectives to optimize are the Average Probability of Dangerous Failure on Demand, which represents the system safety integrity, the Spurious Trip Rate and the Lifecycle Cost. In this way safety, reliability and cost are included. This is done by using novel models of time-dependent probability of failure on demand and spurious trip rate, recently published by the authors. These models are capable of delivering the level of modeling detail required by the standard IEC 61508. Modeling includes common cause failure and diagnostic coverage. The Probability of Failure on Demand model also permits to quantify results with changing testing strategies. The optimization is performed using the multi-objective Genetic Algorithm NSGA-II. This allows weighting of the trade-offs between the three objectives and, thus, implementation of safety systems that keep a good balance between safety, reliability and cost. The complete methodology is applied to two separate case studies, one for optimization of system design with redundancy allocation and component selection and another for optimization of testing policies. Both optimization cases are performed for both systems with MooN redundancies and systems with only parallel redundancies. Their results are compared, demonstrating how introducing MooN architectures presents a significant improvement for the optimization process.

© 2012 Elsevier Ltd. All rights reserved.

1. Introduction

Optimization of *k*-out-of-*n* architectures including maintenance and testing has been previously addressed is several studies. An early analysis for finding the optimal Test Interval (TI) of safety related equipment of Nuclear Power Plants was made by Martorell et al. [1]. It established the average test cycle and quantified unavailability as the contribution of failure between tests, test time and repair time, obtaining simplified analytical equations for the sequential and staggered test strategies of a two-component parallel system. Vaurio [2] developed analytical expressions for quantification of average unavailability for sequential and uniform staggered testing (uniformly distributed over the Test Interval time) for *k*-out-of-*n* systems with up to four components. It included quantification of Common Cause Failure (CCF) by analyzing the convenience of several multi-parameter models. In a subsequent work, Vaurio [3] carried out a study for optimization of test and maintenance intervals at component level (for series systems) and plant level (using fault trees' minimal cut stets) by minimization of cost subject to risk constraints. Martorell et al. [4] approached the interaction of surveillance TI requirements and operating restrictions (i.e., Allowed Outage Time) in Nuclear Power Plants. They identified the unavailability components of the periodic test cycle and developed analytical expressions for each contribution based on probabilistic methods. Cepin and Mavko [5] introduced an unavailability time dependent probabilistic model taking into account the different contributions during the test cycle, and adaptable to several test strategies. The model was later applied in Cepin [6] for optimization of safety equipment outage scheduling due to testing and maintenance by minimization of risk with simulated annealing.

Optimization of *k*-out-of-*n* architectures with genetic algorithms (GA) has been being developed in several other studies. Coit and Smith [7] used a single objective GA for redundancy and component selection: reliability maximization subject to

Abbreviations: CCF, Common Cause Failure; FC, Final Control Subsystem; GA, Genetic Algorithm; IEC, International Electrotechnical Commission; LCC, Lifecycle Cost; LS, Logic Solver; MA, Markov Analysis; MooN, M-out-of-N Voting System; NSGA-II, Non-Dominated Sorting Genetic Algorithm II; PFD, Probability of Failure on Demand; PLC, Programmable Logic Controller; PT, Pressure Transmitter; SIL, Safety Integrity Level; SIS, Safety Instrumented System; STR, Spurious Trip Rate; TI, Test Interval; TS, Test Strategy; TT, Temperature Transmitter

* Corresponding author. Present address: 2500 Woodland Park Dr, Apt C304, Houston, Texas 77077, U.S. Tel.: +1 832 283 5644.
E-mail address: alextorres74@yahoo.com.mx (A.C. Torres-Echeverría).

0951-8320/$ - see front matter © 2012 Elsevier Ltd. All rights reserved.
http://dx.doi.org/10.1016/j.ress.2012.03.010

Nomenclature

CCF_N	Common cause failure of N components
C_{MooN}	β modification factor for MooN system
C_N	sum of all C_{MooN} factor with equal N
C_{test}	constant value of PFD_{ind} during test
C_{TMooN}	C_{MooN} factor modified during test
H_N	β mod. factor for indep. failures of n-comp. syst.
i	ith component being tested or last tested subscript
ij	subsystem index & type index
K	staggering factor
K	number of voting channels
M	number of voting channels
Mod	modulo operator
MooN	M-out-of-N voting system
N, n	number of redundant components or channels
P	test-staggering multiplication factor
$PFD(t)$	time-dependent PFD
$PFD(t_i)$	time-dependent PFD point evaluation
PFD_{avg}	average probability of failure on demand
$PFD_{CCF}(t)$	time-dependent PFD from CCF
$PFD_{ind\ i}(t)$	$PFD(t)$ from ind. failure of the ith component
$PFD_{IND}(t)$	total $PFD(t)$ by combination of independent failures
$PFD_{TOT}(t)$	total system time-dependent PFD
$P_{r\text{-}trip}$	probability of test-induced spurious trip
Psf(CCF)	probability of safe CCF
Psf(SD)	probability of safe detected failure
Psf(SN)	probability of safe normal (independent) failure
Psf(SU)	probability of safe undetected failure
Psf(SUT)	Prob. of s.f. undetected by ε, detected by test
$Q(f)$	probability of failure
Q(Ind)	probability of ind. failure of one component
Rt_{TR}	relative time under test and repair
SFF	safe failure fraction
STR_λ	STR caused by internal failure
$STR_{\lambda\text{-}CCF}$	STR contribution of CCF
$STR_{\lambda\text{-}IND}$	total STR contribution of independent failures
STR_N	STR during normal operation
STR_T	Total spurious trip rate
STR_{test}	proof test-induced STR
STR_{TR}	STR during test and repair
T	Time
TI	test interval
T_{P1}	time to first test of the first component
T_{Pi}	time to first test of the ith component
T_{PN}	time to first test of the Nth component
T_r	repair time
TS	test strategy
T_t	test time
Type	type of device as variable
w_1	test interval remainder of first component
w_i	test interval remainder of the ith component
$\boldsymbol{x}$	decision variables vector
β	β factor for quantification of CCF
β', β_{MooN}	β factor modified for MooN architecture
ε	diagnostic coverage
ε^D	diagnostic coverage for dangerous failures
ε^S	diagnostic coverage for safe failures
ε^{ts}	diagnostic coverage of proof test on safe failures

Failure rates

λ_{test}^{DDC}	dangerous detected CCF during test
λ_{test}^{DUC}	dangerous undetected CCF during test
λ^{STi}	test-induced spurious trip per component
λ^C	CCF rate
λ^D	dangerous failure rate
λ^{DD}	dangerous detected
λ^{DDC}	dangerous detected CCF
λ^{DDN}	dangerous detected normal (independent)
λ^{DU}	dangerous undetected
λ^{DUC}	dangerous undetected CCF
λ^{DUN}	dangerous undetected normal (independent)
λ^N	normal (independent)
λ^S	safe failure rate
λ^{SC}	safe CCF rate
λ^{SD}	safe detected
λ^{SDC}	safe detected CCF
λ^{SDN}	safe detected normal (independent)
λ^{SN}	safe normal (independent)
λ^T	total failure rate
λ^{SU}	safe undetected
λ^{SUC}	safe undetected CCF
λ^{SUN}	safe undetected normal (independent)
λ^{SUTC}	safe undetected by ε, detected by test (CCF)
λ^{SUTN}	safe undetected by ε, detected by test (indep.)
λ^{SYS}	total system failure rate

Lifecycle cost model (Eqs. ((22)–(33)))

C_{ACC}	cost of accident
C_{CM}, C^{CM}	corrective maintenance cost
C_D	design cost
C_{HAZARD}	cost per accident event
$C^{inst/comm}$	installation and commissioning cost
C_{OP}	operation cost
C_{PM}, C^{PM}	preventive maintenance cost
C_{PROC}	procurement cost
$C^{purchase}$	acquisition cost
C_{RISK}	risk cost
C_{SD}	cost per spurious shutdown event
C_{STR}	cost of spurious trip rate
C_T, C^T	proof test cost, proof test cost per event
$F(ACC\|PFD_{avg})$	accident frequency without SIS per year
f^{CM}	corrective maintenance frequency
LCC	Lifecycle cost
M_{ij}	preventive maintenance frequency
N_{ij}	component of jth type of the ith subsystem
PVF	present value factor
R	discount rate
ij	subsystem index & type index
SD_{LOSS}	cost of production loss per hour
SD_{Time}	restart time after shutdown
T	total period evaluation time (mission time)
TI_{ij}	test interval of component ij
λ^T	total failure rate

cost and weight constraints. The problem is optimization of systems where k-out-of-n:G subsystems can be chosen. They are solved by the binomial distribution formula. In a subsequent work [8] they estimated the quantification of reliability of a large system with k-out-of-n redundancies using neural networks.

Pattison and Andrews [9] implemented the single objective GA optimization of a High Integrity Protection System with component selection, and where the number n of transmitters per subsystem and the number k necessary to trip were decision variables. The system dependability is estimated by using fault trees synthesized by Binary Decision Diagrams. The single-objective optimization is for system unavailability with cost, STR and maintenance time constraints. The methodology was later scaled up for the larger problem of the firewater deluge system [10]. This work was later adapted to multi-objective optimization in Ref. [11].

Cantoni et al. [12] combined a single objective GA with Monte Carlo simulation for optimization of design of a series-parallel system with k-out-of-n:G and cold-standby architecture options (redundancy and components selection). This was later extended to multi-objective optimization [13], where a nuclear reactor protective system is analyzed, which contains a 2oo4 system modeled by MA. The optimization is not for plant design but for test and maintenance policies. Mishra et al. [14] focused on two-objective optimization using a multi-objective GA. The optimization was at two levels, component and plant level. The application was made for optimization of maintenance and testing related parameters (called technical specifications) against cost, in two separate cases, one related to unavailability of safety-critical systems (optimization at system level), and the second to risk of accident events that can damage the reactor's core of a nuclear power plant (optimization at plant level). They included both standby and continuous-operating safety systems, with inclusion of extensive parameters in both models. Tavakkoli et al. [15] proposed single-objective reliability optimization by GA, where the redundancy strategy of each individual subsystem can be selected between active and cold k-out-of-n standby schemes.

Torres-Echeverria et al. [16–18] developed models for quantification of PFD(t) and STR for systems with parallel architectures with sufficient detail to meet the quantification requirements of IEC 61508 (including CCF, diagnostic coverage, redundancy levels and test intervals and strategies), with the aim of making them suitable for optimization of design and test strategies using metaheuristic methods. The models were applied for optimization of safety systems using genetic algorithms. The cases applied were optimization of design of safety systems with parallel redundancies [16], optimization of design with diverse parallel redundancy [17], and optimization of safety system testing strategies [18]. Kancev et al. [19] presented a multi-objective GA approach for optimization of surveillance test intervals of an standard nuclear industry standby safety system. The approach is based on minimization of system unavailability and aging data uncertainty impact. The study makes a novel contribution by integrating component aging together with effects of test & maintenance and testing strategies as decision variables. The component aging uncertainty is estimated using an analytical unavailability model together with Monte Carlo simulation.

The work presented in this paper is intended to apply the time-dependent probability of failure on demand (PFD) and spurious trip rate (STR) models developed recently by the authors [20] to multi-objective optimization by GA of design and testing of Safety Instrumented Systems (SIS) with MooN redundancies in their subsystems. The developments presented in previous articles by the authors have addressed redundant systems implemented with only parallel redundancies (optimization of safety system design [16,17] and testing policies [18]). Ref. [20] extended the previous models to handling MooN voting systems included in the system architecture. This article now approaches the optimization of Safety Instrumented Systems (SIS) where inclusion of MooN voting architectures is an available option for subsystem redundancy. It includes optimization of system design and optimization of test policies. The main aim is to explore whether introducing voting architectures as an alternative to simple parallel redundancies allow achievement of a better balance between the system's dependability attributes, average PFD (PFD_{avg}) and STR, and therefore better lifecycle cost (LCC). Regarding optimization of the testing policies, this paper also analyzes if introduction of MooN subsystems in the system facilitates the implementation of better testing polices that counteract the negative impact of testing while still giving benefits in terms of safety and cost.

The structure of the paper is as follows: First, a preliminary review of concepts related to MooN systems is provided together with a review of their application in optimization cases in Section 2. Subsequently, Section 3 introduces the more basic requirements of the standard IEC 61508 and the theory behind failure modes classification. Section 4 includes a description of the PFD model, while Section 5 does the same for the STR model. The model for quantification of LCC is described in Section 6. Section 7 presents a brief description of the NSGA-II multi-objective GA. Afterwards the models are then integrated with the GA to produce the optimization of SIS, which application is illustrated in Section 8 for system design, and Section 9 for testing policies. The conclusions of this paper are then presented in Section 10.

2. Effects of using MooN architectures

A MooN system is defined as a system with N units (i.e., components, channels, etc.) in which M out of them are sufficient to initiate the safety function. It requires a minimum of M units to "vote" for the execution of the safety function. It also implies that these M units must be functional for the system to be successful, and that a voting mechanism is provided. This is equivalent to a k-out-of-n:G with an added voter.

Safety systems are required to provide fault tolerance, which is fundamental against dangerous failures. Fault tolerance is implemented by using redundancy. The most basic and used form of redundancy is the parallel structure. However, every component that is added in parallel to the system introduces a new chance of failure. This results in new chances of spurious trips, which can bear a considerable cost. This is a clear manifestation of the conflict between PFD and STR. Looking for a solution to this conflict, voting systems are introduced in the search for the achievement of a better compromise between fault tolerance and spurious trip rate.

It can be seen that a voting system permits implementation of fault tolerance with a lower STR than a parallel system with the same number of components N. Although this improves the PFD_{avg} compared with a single-component system, compared with a similar parallel system it, however, raises it. Nevertheless, it still contributes to ensuring continuous operation of the safety system.

Considering this, the option of choosing amongst voting systems could permit an additional reduction of STR while implementing fault tolerance. This may represent an increment in the acquisition cost, but a decrement of STR costs. It is thus a question of cost balance to achieve a positive impact on the system LCC. Due to the increment on PFD_{avg} (in comparison to a similar parallel system), a voting architecture could have a negative impact on the test intervals. Also notice that proof testing may also have a negative impact on STR [18]. This work, therefore, seeks to determine whether the benefits of introducing voting architectures regarding STR outweigh the disadvantages of higher PFD_{avg} (in terms of LCC), and if the necessary increment in testing activity is compensated.

Consider in addition the impact of CFF on both system performance measures and costs. Implementing redundancy

implies that CFF is introduced, which is another factor that has an impact on the system's three main objectives: PFD_{avg}, STR and indirectly LCC. The PDS Method [21] shows that, for a fixed number of components N, as the required number of voting components M needed to initiate the safety function increases the voting level-dependent configuration factor C_{MooN} (see Eq. (4) below) increases as well (i.e., $C_{1oo3} < C_{2oo3}$, $C_{1oo4} < C_{2oo4} < C_{3oo4}$). This change is in the opposite direction for STR. Therefore, the impact of introducing voting into system optimization becomes less clear. This is another feature researched herein.

3. Sis requirements and failure mode classification

3.1. Requirements of IEC 61508

The international standard IEC 61508 [22] addresses the effective achievement of safety when this is based on the functionality of safety-related systems. SIS fall within this category. Therefore, the design of SIS is subject to achievement of some requirements established by IEC 61508 regarding safety integrity. These safety integrity requirements include the restriction of the system PFD_{avg} to a maximum target limit and the compliance with some minimum levels of fault tolerance.

One of the most salient features of IEC 61508 is the definition of Safety Integrity Levels (SIL). The standard requires that every safety function must achieve a specific SIL, which is determined based on a previous risk assessment. The SIL is a quantitative index that indicates the acceptable probability of dangerous failure that a system can have to consider it appropriate for a given specific safety integrity requirement. Distinction is made between two different types of systems: low demand mode and high demand/continuous mode of operation. SIS are most frequently implemented in low demand mode of operation. For these, the SIL levels are defined in terms of average probability of failure on demand (Table 1). In addition to the requirement of bounded PFD_{avg}, the highest SIL that can be claimed for a subsystem's combination for hardware is limited by architectural constraints, which are detailed in IEC 61508 Part 2. The interested reader can seek a more detailed account in Ref. [17], or directly in the standard.

3.2. Failure modes classification

SIS can fail in two different ways: Dangerous and safe. IEC 6108 defines dangerous failures (λ^D) as those "*which has the potential to put the safety-related system in a hazardous or fail-to-function state*". In other words, they are those failures that prevent the system to perform its intended function on a real demand to do so. In contrast, safe failures (λ^S) are those which do not threat the ability of the safety system to perform its intended function. However, they can lead the system to operate without an actual demand; i.e., spurious operation. Thus, it can be concluded that the dangerous failures contribute to the system PFD_{avg}, while, in contrast, the safe failures contribute to the system STR. Additionally, the system built-in diagnostic coverage (ε) divides both dangerous and safe fractions into detected (λ^{DD} and λ^{SD}) and undetected (λ^{DU} and λ^{SU}) failure modes (Eq. (1)).

Table 1
SIL for low demand mode of operation [22].

SIL	PFD_{avg}
4	$\geq 10^{-5}$ to $< 10^{-4}$
3	$\geq 10^{-4}$ to $< 10^{-3}$
2	$\geq 10^{-3}$ to $< 10^{-2}$
1	$\geq 10^{-2}$ to $< 10^{-1}$

$$\lambda^T = \varepsilon\cdot\lambda^D + (1-\varepsilon)\cdot\lambda^D + \varepsilon\cdot\lambda^S + (1-\varepsilon)\cdot\lambda^S = \lambda^{DD} + \lambda^{DU} + \lambda^{SD} + \lambda^{SU} \quad (1)$$

The phenomenon of CCF further divides the failure rates into independent (λ^N, where N stands for "normal") and common cause (λ^C). These fractions can be quantified by the β factor method [23]. The β factor represents the fraction of the total failure rate that can be attributed to CCF.

$$\lambda^T = (1-\beta)\lambda^T + \beta\lambda^T = \lambda^N + \lambda^C \quad (2)$$

Applying the fractions determined by the β factor (Eq. (2)), the total failure rate is split into eight different contributions.

$$\lambda^T = \lambda^{DDN} + \lambda^{DDC} + \lambda^{DUN} + \lambda^{DUC} + \lambda^{SDN} + \lambda^{SDC} + \lambda^{SUN} + \lambda^{SUC} \quad (3)$$

The PDS Method [21] formulates an extension to the β factor model that enables the β factor to be modified according to the voting logic MooN of the system by a configuration-specific modification factor C_{MooN} (Eq. (4)). This method is used herein for quantification of CCF. The modified β' can be applied to Eq. (2) to determine the common cause and independent failure rates.

$$\beta' = \beta_{MooN} = \beta\cdot C_{MooN} \quad (4)$$

4. Modeling PFD for MooN architectures

This section outlines in a succinct way the time-dependent PFD model presented in Ref. [20]. The full analysis behind the development of the model can be found in that reference. The model is presented here for the sake of completeness. This model has the power to explicitly model CCF, diagnostic coverage and changes in the mean test cycle of the components. It therefore allows handling the level of detail required by IEC 61508.

This article assumes that SIS are configured as de-energize-to-trip systems (the system de-energizes the components for execution of the safety function) and that the components are open-to-trip (a switch will open to execute the safety action). Also, remember that for a MooN architecture to execute the safety function M channels (or components) must agree to de-energize the system output. Since the components of a safety system need to be bypassed in order to be available for testing, a bypassing philosophy has to be formulated. The philosophy used here is described as "continue production during testing with degraded operation in the safest possible way". Therefore, the bypassing strategy to be used is: Bypass closed for 1ooN architectures, and bypass open for MooN architectures ($M > 1$).

The total time dependent PFD of a MooN architecture ($PFD_{TOT}(t)$) is the sum of the PFD produced by the combination of independent failures ($PFD_{IND}(t)$) plus the PFD by CFF:

$$PFD_{TOT}(t) = PFD_{IND}(t) + PFD_{CCF}(t) \quad (5)$$

The time-dependent PFD model is presented fully in Fig. 1(a) and (b). This model handles instant quantification of PFD at component level. The contribution of CCF to PFD(t) is modeled as a single component, and the CCF rates are quantified using the appropriate C_{MooN} modification factor.

The failure rates of the different failure modes to be fed into the PFD(t) are calculated as given by Eqs. (6)–(11). The constant C_{Test} and the appropriate modification factors to apply are indicated in Table 2.

$$\lambda^{DDN} = \varepsilon\cdot(1-H_N\beta)\lambda^D \quad (6)$$

$$\lambda^{DUN} = (1-\varepsilon)\cdot(1-H_N\beta)\lambda^D \quad (7)$$

$$\lambda^{DDC} = \varepsilon\cdot\beta\cdot C_{MooN}\cdot\lambda^D \quad (8)$$

a

Normal operation before first test:

$$PFD_{ind\,i}(t) = \lambda^{DDN} T_r + 1 - e^{-\lambda^{DUN} t} \quad \text{for} \quad t - T_{Pi} < 0$$

Testing:

$$PFD_{ind\,i}(t) = C_{Test} \quad \text{for} \quad \{t - T_{Pi} \geq 0 \;\&\; 0 \leq w_i < T_t\}$$

Repair:

$$PFD_{ind\,i}(t) = \begin{cases} \lambda^{DDN} T_r + 1 - e^{-\lambda^{DUN} T_{Pi}} + (e^{-\lambda^{DUN} T_{Pi}} - \lambda^{DDN} T_r)(\lambda^{DDN} T_r + 1 - e^{-\lambda^{DUN}(w_i - T_t)}) \\ \quad \text{for } \{0 \leq t - T_{Pi} < TI \;\&\; T_t \leq w_i < T_t + T_r\} \text{ (first repair)} \\ \lambda^{DDN} T_r + 1 - e^{-\lambda^{DUN}(TI - T_t - T_r)} + (e^{-\lambda^{DUN}(TI - T_t - T_r)} - \lambda^{DDN} T_r)(\lambda^{DDN} T_r + 1 - e^{-\lambda^{DUN}(w_i - T_t)}) \\ \quad \text{for } \{t - T_{Pi} \geq TI \;\&\; T_t \leq w_i < T_t + T_r\} \text{ (subsequent repairs)} \end{cases}$$

Normal operation between tests:

$$PFD_{ind\,i}(t) = \lambda^{DDN} T_r + 1 - e^{-\lambda^{DUN}(w_i - T_t - T_r)} \quad \text{for} \quad \{t - T_{Pi} \geq 0 \;\&\; w_i \geq T_t + T_r\}$$

where:

$$w_i = (t - T_{Pi}) \bmod TI$$

b

Normal operation before first test:

$$PFD_{CCF}(t) = \lambda^{DDC} T_r + 1 - e^{-\lambda^{DUC} t} \quad \text{for} \quad t - T_{P1} < 0$$

Testing:

$$IF \quad |T_{Pi+1} - T_{Pi}| < T_t \quad \text{(Simultaneous test)}$$

$$PFD_{CCF}(t) = 0 \quad \text{for} \quad \left\{ t - T_{P1} \geq 0 \;\&\; \begin{vmatrix} 0 \leq w_1 < T_t \\ T_{Pi} - T_{P1} \leq w_1 < T_{Pi} - T_{P1} + T_t \end{vmatrix} \right\}$$

$$IF \quad T_{Pi+1} - T_{Pi} \geq T_t \quad \text{(Non- simultaneous test)}$$

$$PFD_{CCF}(t) = \begin{cases} \lambda_{Test}^{DDC} T_r + 1 - e^{-\lambda_{Test}^{DUC} t} \\ \quad \text{for } \{0 \leq t - T_{P1} < TI \quad \& \quad 0 \leq w_1 < T_t\} \text{ (first test of cycle for initial state)} \\ \lambda_{Test}^{DDC} T_r + 1 - e^{-\lambda_{Test}^{DUC}(w_1 - (T_{Pi-1} - T_{P1}) - T_t - T_r)} \\ \quad \text{for } \{0 \leq t - T_{P1} \quad \& \quad T_{Pi} - T_{P1} \leq w_1 < T_{Pi} - T_{P1} + T_t\} \text{ (any other test)} \\ \lambda_{Test}^{DDC} T_r + 1 - e^{-\lambda_{Test}^{DUC}(w_1 + TI - (T_{PN} - T_{P1}) - T_t - T_r)} \\ \quad \text{for } \{t - T_{P1} \geq TI \quad \& \quad 0 \leq w_1 < T_t\} \text{ (first test of cycle for steady state)} \end{cases}$$

Repair:

$$PFD_{CCF}(t) = \begin{cases} \lambda^{DDC} T_r + 1 - e^{-\lambda^{DUC} T_{P1}} + (e^{-\lambda^{DUC} T_{P1}} - \lambda^{DDC} T_r)(\lambda^{DDC} T_r + 1 - e^{-\lambda^{DUC}(w_1 - T_t)}) \\ \quad \text{for } \{0 \leq t - T_{P1} < TI \quad \& \quad T_t \leq w_1 < T_t + T_r\} \text{ (first repair of cycle for initial state)} \\ \lambda^{DDC} T_r + 1 - e^{-\lambda^{DUC}(T_{Pi} - T_{Pi-1} - T_t - T_r)} + (e^{-\lambda^{DUC}(T_{Pi} - T_{Pi-1} - T_t - T_r)} - \lambda^{DDC} T_r)(\lambda^{DDC} T_r + 1 - e^{-\lambda^{DUC}(w_1 - (T_{Pi} - T_{P1}) - T_t)}) \\ \quad \text{for } \{t - T_{P1} \geq 0 \quad \& \quad T_{Pi} - T_{P1} + T_t \leq w_1 < T_{Pi} - T_{P1} + T_t + T_r\} \\ \lambda^{DDC} T_r + 1 - e^{-\lambda^{DUC}(TI - (T_{PN} - T_{P1}) - T_t - T_r)} + (e^{-\lambda^{DUC}(TI - (T_{PN} - T_{P1}) - T_t - T_r)} - \lambda^{DDC} T_r)(\lambda^{DDC} T_r + 1 - e^{-\lambda^{DUC}(w_1 - T_t)}) \\ \quad \text{for } \{t - T_{P1} \geq TI \quad \& \quad T_t \leq w_1 < T_t + T_r\} \text{ (first repair of cycle for steady state)} \end{cases}$$

Normal operation between tests:

$$PFD_{CCF}(t) = \lambda^{DDC} T_r + 1 - e^{-\lambda^{DUC}(w_1 - (T_{Pi} - T_{P1}) - T_t - T_r)}$$

$$\text{for} \left\{ t - T_{P1} \geq 0 \;\&\; \begin{vmatrix} T_{Pi} - T_{P1} + T_t + T_r \leq w_1 < T_{Pi+1} - T_{P1} & \text{if} \quad i < N \\ T_{PN} - T_{P1} + T_t + T_r \leq w_1 < T_t & \text{if} \quad i = N \end{vmatrix} \right\}$$

where:

$$w_1 = (t - T_{P1}) \bmod TI$$

Fig. 1. PFD(t) model for MooN architectures (a) independent failures contribution. (b) Common cause failures contribution.

Table 2
Modification factors.

Architecture	C_{Test}	C_{MooN}	C_{TMooN}
1ooN	1	C_{1ooN}	$C_{1oo(N-1)}$
MooN ($1 < M < N$)	0	C_{MooN}	C_{MooN}
NooN	0	$C_{(N-1)ooN}$	$C_{(N-1)ooN}$

$$\lambda^{DUC} = (1-\varepsilon)\cdot\beta\cdot C_{MooN}\cdot\lambda^{D} \tag{9}$$

$$\lambda_{test}^{DDC} = \varepsilon\cdot\beta\cdot C_{TMooN}\cdot\lambda^{D} \tag{10}$$

$$\lambda_{test}^{DUC} = (1-\varepsilon)\cdot\beta\cdot C_{TMooN}\cdot\lambda^{D} \tag{11}$$

There are some differences between the operation and test states in the quantification of PFD. The contribution during test of $PFD_{ind\,i}(t)$ is equal to a constant C_{test} (Fig. 1(a)) that takes a value that depends on how it is bypassed during test (closed=1, open=0), as indicated in Table 2. On the other hand, CCF rates are quantified based on distinct modification factors (Fig. 1(b)). There are two factors, C_{MooN} and C_{TMooN}, one for normal operation and one for test, respectively. These change according to the MooN architecture being modeled as indicated in Table 2. Also notice that the CCF failure rates during test are indicated as λ_{test}^{DDC} and λ_{test}^{DUC} in the model given in Fig. 1(b).

Table 3 reproduces the table given by the PDS Method for the values of C_{MooN}, C_N and H_N of MooN architectures up to $N=6$. These are suggested example values. Notice that the β factor used in Eqs. (6)–(11) is the basic β used for two components (i.e., 1oo2), and that $C_{1oo2}=1$.

Table 4 formulates the combinations of both PFD_{IND} and PFD_{CFF} at MooN system level. The formulation presented in this table can be used to determine how to adapt the PFD(t) component-level-model for modeling MooN systems. The combinatorial expressions presented in Table 4 are the basis to determine the $PFD_{TOT}(t)$. These determine how a fault tree can be created for quantification of the total PFD of a specific MooN system. The column headed P_{IND} (for normal operation) guides the construction of fault trees related to the contribution of the independent failures of each component as follows: The coefficients of each formula determine the number of combinations of independent failures. The exponents indicate the number of basic events (independent failures) to be included per combination. The basic events of the tree would be quantified by the equations of the PFD(t) model (Fig. 1(a) and (b)).

5. Modeling STR for MooN architectures

This section outlines the model presented by Ref. [20] for quantification of STR. The reader is encouraged to consult this source to understand the development of the model. As established earlier, the bypassing strategy is that the bypass will be closed for 1ooN and open for MooN ($M>1$) architectures. Since dangerous and safe failures are opposite failure modes, the bypasses take the opposite value in the PFD and the STR fault trees. For STR a closed bypass is equivalent to a value of zero, and an open bypass to a value of one.

The total STR of a MooN system is the sum of the STR attributed to internal failure (STR_λ), and the test induced (STR_{test}):

$$STR_T = STR_\lambda + STR_{test} = STR_{\lambda\text{-IND}} + STR_{\lambda\text{-CCF}} + STR_{test} \quad (12)$$

Table 5 summarizes the formulae for quantification of STR for different MooN architectures.

In the table λ^{STi} stands for test-induced spurious trips failure rate per component, which is quantified by:

$$\lambda^{STi} = \frac{1}{TI}\cdot P_{r\text{-trip}} \quad (13)$$

Notice that the STR_λ of a determined MooN architecture changes during test as a consequence of the system being reconfigured by the bypass of the component under test. The relative time spent in normal operation and with one component under test (as determined by Ref. [24]) is expressed as:

$$STR_\lambda = (1-Rt_{TR})\cdot STR_N + Rt_{TR}\cdot STR_{TR} \quad (14)$$

where:

$$Rt_{TR} = \frac{T_t + T_r\cdot Q(f)}{TI} \quad (15)$$

where $Q(f)$ is the probability of the component under test being found failed, and which complete expression is:

$$Q(f) = (\lambda^{DDN} + \lambda^{DDC} + \lambda^{SDN} + \lambda^{SDC})T_r + 1 - e^{-(\lambda^{DUN} + \lambda^{DUC} + \lambda^{SUTN} + \lambda^{SUTC})TI} \quad (16)$$

where:

$$\lambda^{SUTN} = \lambda^{SUN}\cdot\varepsilon^{ts} \quad (17)$$

$$\lambda^{SUTC} = \lambda^{SUC}\cdot\varepsilon^{ts} \quad (18)$$

Regarding the quantification of STR using fault trees, it is necessary to convert some safe failure rates into probability of safe failures, given that two or more failure-rate basic events cannot be multiplied in AND gates. The probability of safe failures is split into three different failure modes given by Eqs. (19)–(21). For safe detected (SD) failures (assuming they are detected and repaired immediately) the probability of safe failure (Psf) is determined by the fault residence time, in this case the T_r (Eq. (19)). Although usually proof testing is aimed at targeting mainly dangerous failures, if we assume that it actually manage to detect some fraction of safe failures, an intermediate failure mode between detected and undetected failures appears: the safe failures undetected by the automatic diagnostic coverage but detected by proof test (SUT), given by Eq. (20), where ε^{ts} represents the proof test diagnostic coverage on safe failures.

$$Psf(SD) = \lambda^S\cdot\varepsilon\cdot T_r = \lambda^{SD}\cdot T_r \quad (19)$$

$$Psf(SUT) = \lambda^S\cdot(1-\varepsilon)\cdot\varepsilon^{ts}\cdot(TI+T_r) = \lambda^{SU}\cdot\varepsilon^{ts}\cdot(TI+T_r) \quad (20)$$

$$Psf(SU) = \lambda^S\cdot(1-\varepsilon)\cdot(1-\varepsilon^{ts})\cdot T = \lambda^{SU}\cdot(1-\varepsilon^{ts})\cdot T \quad (21)$$

These failure modes can be split each into independent and common cause failure fractions, using the right C_{MooN} factor (indicated in the last column of Table 5).

Table 3
Values of C_{MooN}, C_N and H_N [21].

	C_{MooN}						
	$M=1$	$M=2$	$M=3$	$M=4$	$M=5$	C_N	H_N
$N=2$	1.0	–	–	–	–	1.0	1.0
$N=3$	0.30	2.4	–	–	–	2.7	1.7
$N=4$	0.15	0.75	4.0	–	–	4.9	2.2
$N=5$	0.08	0.45	1.2	6.0	–	7.7	2.7
$N=6$	0.04	0.26	0.8	1.6	8.1	10.8	3.2

6. Lifecycle cost quantification

The system LCC is the third objective to optimize, which comprises all procurement, operation and risk costs. LCC model is presented in Eqs. (22)–(33). The model is taken from

Table 4
Combinations of PFD at MooN system level.

Architecture	State	PFD_{IND}	PFD_{CCF}	C_{MooN}
1ooN	Normal operation	$Q(\text{Ind})^N$	$Q(CCF_N)$	C_{1ooN}
	Test	$Q(\text{Ind})^{N-1}$	$Q(CCF_{N-1})$	$C_{1oo(N-1)}$
MooN ($1<M<N$)	Normal operation	$\binom{N}{N-M+1}\cdot Q(\text{Ind})^{N-M+1}$	$Q(CCF_{MooN})$	C_{MooN}
	Test	$\binom{N-1}{N-M+1}\cdot Q(\text{Ind})^{N-M+1}$	$Q(CCF_{MooN})$	C_{MooN}
NooN	Normal operation	$N\cdot Q(\text{Ind})$	$Q(CCF_{(N-1)ooN})$	$C_{(N-1)ooN}$
	Test	$(N-1)\cdot Q(\text{Ind})$	$Q(CCF_{(N-1)ooN})$	$C_{(N-1)ooN}$

Table 5
Summary of formulae for quantification of STR.

Architecture	State	STR	Contribution	C_{MooN}
1ooN	N.O.	$STR_{\lambda\text{-IND}}$	$N\cdot\lambda^{SN}$	$C_{(N-M)\text{ooN}}$
		$STR_{\lambda\text{-CCF}}$	$Psf(CCF_{N-M+1})$	
	Test	$STR_{\lambda\text{-IND}}$	$(N-1)\cdot\lambda^{SN}$	$C_{(N-M)\text{ooN}}$
		$STR_{\lambda\text{-CCF}}$	$Psf(CCF_{N-M+1})$	
		STR_{test}[a]	$N\cdot\lambda^{STi}$	
2ooN	N.O.	$STR_{\lambda\text{-IND}}$	$\binom{N}{M}\cdot\lambda^{SN}\cdot Psf(SN)$	$C_{(N-M+1)\text{ooN}}$
		$STR_{\lambda\text{-CCF}}$	$Psf(CCF_{N-M+1})$	
	Test	$STR_{\lambda\text{-IND}}$	$\left[(N-1)+\left(\binom{N}{M}-(N-1)\right)Psf(SN)\right]\cdot\lambda^{SN}$	$C_{(N-M+1)\text{ooN}}$
		$STR_{\lambda\text{-CCF}}$	$Psf(CCF_{N-M+1})$	
		STR_{test}[a]	$\left[(N-1)+\left(\binom{N}{M}-(N-1)\right)P_{r-trip}\right]\cdot\lambda^{STi}$	
MooN ($2<M<N$)	N.O.	$STR_{\lambda\text{-IND}}$	$\binom{N}{M}\lambda^{SN}\cdot Psf(SN)^{M-1}$	$C_{(N-M+1)\text{ooN}}$
		$STR_{\lambda\text{-CCF}}$	$Psf(CCF_{N-M+1})$	
	Test	$STR_{\lambda\text{-IND}}$	$\left[\binom{N-1}{M-1}Psf(SN)^{M-2}+\left(\binom{N}{M}-\binom{N-1}{M-1}\right)Psf(SN)^{M-1}\right]\cdot\lambda^{SN}$	$C_{(N-M+1)\text{oo}(N-1)}$
		$STR_{\lambda\text{-CCF}}$	$Psf(CCF_{N-M+1})$	
		STR_{test}[a]	$\left[\binom{N-1}{M-1}(P_{r-trip})^{M-2}+\left(\binom{N}{M}-\binom{N-1}{M-1}\right)(P_{r-trip})^{M-1}\right]\cdot\lambda^{STi}$	
NooN	N.O.	$STR_{\lambda\text{-IND}}$	$\lambda^{SN}\cdot Psf(SN)^{N-1}$	$C_{1\text{ooN}}$
		$STR_{\lambda\text{-CCF}}$	$Psf(CCF_N)$	
	Test	$STR_{\lambda\text{-IND}}$	$\lambda^{SN}\cdot Psf(SN)^{N-2}$	$C_{1\text{oo}(N-1)}$
		$STR_{\lambda\text{-CCF}}$	$Psf(CCF_{N-1})$	
		STR_{test}[a]	$\lambda^{STi}\cdot(P_{r-trip})^{N-2}$	

N.O.=Normal Operation.

[a] The STR_{test} applies only during test (during normal operation $STR_{\text{test}}=0$).

Refs. [17,18]. The overall lifecycle cost is given by:

$$LCC = C_{\text{PROC}} + (C_{\text{OP}} + C_{\text{RISK}})\cdot PVF \tag{22}$$

where the applied factor by present value is:

$$PVF = \frac{1-(1+R)^{-T}}{R} \tag{23}$$

Procurement cost per year is quantified as:

$$C_{\text{PRO}} = C_D + \sum_{\forall ij}(C_{ij}^{\text{purchase}} + C_{ij}^{\text{inst/comm}})N_{ij} \tag{24}$$

where N_{ij} is the number of components of jth type of the ith subsystem.

The overall operation cost per year (Eq. (25)) includes preventive maintenance, testing and corrective maintenance (Eqs. (26)–(28)).

$$C_{OP} = C_{PM} + C_T + C_{CM} \tag{25}$$

$$C_{PM} = \sum_{\forall ij}\frac{1}{M_{ij}}C_{ij}^{PM}\cdot N_{ij} \tag{26}$$

$$C_T = \sum_{\forall ij}\frac{1}{TI_{ij}}C_{ij}^{T}\cdot N_{ij} \tag{27}$$

$$C_{CM} = \sum_{\forall ij} f_{ij}^{CM}\cdot C_{ij}^{CM}\cdot N_{ij} \tag{28}$$

where:

$$\sum_{\forall ij} f_{ij}^{CM} = \sum_{\forall ij}\lambda_{ij}^{T} \tag{29}$$

The risk cost per year comprises both the cost per spurious trips and the cost of hazard (related to dangerous failures):

$$C_{\text{RISK}} = C_{\text{STR}} + C_{\text{HAZARD}} \tag{30}$$

The cost for spurious trip rate is basically directly proportional to the cost production loss by spurious shutdown:

$$C_{\text{STR}} = STR\cdot C_{\text{SD}} \tag{31}$$

where:

$$C_{\text{SD}} = SD_{\text{Time}}\cdot SD_{\text{LOSS}} \tag{32}$$

The cost of hazard includes plant assets loss, deferred production and liability costs:

$$C_{\text{HAZARD}} = C_{\text{ACC}}\cdot F(\text{ACC}|\text{PFD}_{\text{avg}})\cdot \text{PFD}_{\text{avg}} \tag{33}$$

where the $F(\text{ACC}|\text{PFD}_{\text{avg}})$ is the frequency of plant failure (or accident) per year without the SIS.

Notice that Eq. (30) considers the impact that proof testing has upon the plant's risk cost. Proof testing has both positive and negative effects upon LCC, since the practice of periodic proof testing reduces the dangerous failure risk when improving PFD_{avg}, but it can increment the safe failure risk (through its adverse impact on the STR).

7. Multi-objective GA based on NSGA-II

Genetic Algorithms pertain to the evolutionary computation techniques. GAs mimic the natural selective process of evolution for solution of optimization problems. They present the advantages that they are very useful for solving complex, high dimensional, discrete, non-linear and discontinuous problems, with capacity to handle integer variables. GAs are able to deal with problems which objective function is not explicit, such as those generated by fault tree analysis. In addition, they provide the decision maker a pool of

optimal solutions. There are many studies related to GAs, since it is a research field in continuous evolution. A good study of GAs can be found in Deb [25]. For GAs focussed on RAMS optimization refer to Marseguerra et al. [26] and Konak et al. [27]. For GA-based optimization of testing Martorell et al. [28,29] and Villanueva et al. [30] can be consulted.

GAs work with an initial population of potential solutions called individuals. Each individual is a potential solution to the optimization problem: a coded representation of one set of decision variables in the decision space. These individuals are evaluated, selected and mated to create new and hopefully better ones, which are fed into a new generation, making an iterative process that mimics the natural evolution. Each step of the GA is executed through the application of genetic operators.

The standard GA follows generally the following steps:

1. Creation of a random initial population
2. Evaluation of the individuals, which are ranked and fitness-assigned
3. Selection for reproduction (crossover), generating a number of new individuals (offsprings)
4. Mutation of the offsprings
5. Replacement of the offsprings into the original population.

The cycle is repeated until a certain condition to stop the algorithm is met. This condition may be the exhaust of a generation count or the compliance with a specific objective. At the end, the algorithm delivers a set of optimal solutions: the Pareto-optimal front.

The NSGA-II [31] is one of the most competitive algorithms developed so far. The original algorithm was later further improved with the feature of controlled elitism [32]. The NSGA-II is a second-generation algorithm. The characteristic feature of this generation is the implementation of elitism in order to preserve the best individuals for subsequent generations. The NSGA-II has the following main features:

- Ranking based on non-domination sorting
- Crowding distance metric as explicit diversity-preserving mechanism
- Implementation of elitism
- It does not use an external population (as other second-generation algorithm do)
- Optional controlled elitism.

Ref. [18] presented a description of the algorithm and the controlled elitism mechanism. This is reproduced here in order to facilitate the reader the implementation of the algorithm. A flowchart describing the NSGA-II is shown in Fig. 2. The algorithm works with a population of fixed size N_{pop}. After creating a random initial parent population $\mathbf{P_0}$, an offspring population $\mathbf{Q_0}$, is created based on binary tournament selection. Both populations are combined in a population $\mathbf{R}$ (size $2 \cdot N_{pop}$). Ranking is then carried out by sorting the individuals by non-domination, creating several different non-dominated fronts. A completely new parent population $\mathbf{P}$ is generated by adding the individuals from the fronts, starting progressively from the first front (better rank), and so on until a spillover occurs ($\mathbf{P} > N_{pop}$). This is what enforces the elitist mechanism. The entire population of the last front (which spilled over) is sorted by the crowding distance metric (the diversity-preserving mechanism), and the individuals with wider spread (larger crowding distance) are inserted into $\mathbf{P}$ only until $\mathbf{P}=N_{pop}$. This gives as a result the new parent population $\mathbf{P}$. From these a new offspring population $\mathbf{Q}$ is created by crowded tournament selection. By this procedure two solutions are compared, and the one with better rank is chosen; if both have the same rank the one with better crowding distance metric is picked. The new populations $\mathbf{P}$ and $\mathbf{Q}$ are then used to feed a new generation (to make $\mathbf{R}=\mathbf{P}+\mathbf{Q}$). The whole procedure is repeated until the termination criterion is met (in Fig. 2 this is a pre-established maximum number of generations).

The emphasis over elite solutions made twice by NSGA-II can result in solutions not belonging to non-elitist fronts being deleted too soon. This compromises the balance of exploitation vs exploration. This was noticed by Deb and Goel [32]. They identified that in order to preserve the good balance, it is necessary to ensure good diversity both along the Pareto-optimal front and also lateral to the front. This latter is called lateral diversity. Deb and Goel conceived a mechanism that intends "*to maintain a predefined distribution of the number of individuals in each front*" (not only the elitist ones), and thus foster lateral diversity (at the time the NSGA-II promotes diversity along the front). This is made by restricting "*the number of individuals in the current best non-dominated front adaptively*". Controlled elitism is introduced into the NSGA-II algorithm for generating a new parent population $\mathbf{P}$ during each generation. The right side of Fig. 2 shows the controlled elitism algorithm flowchart. The figure also indicates how the controlled elitism is introduced into the original NSGA-II algorithm. The parameter r (which is <1) helps to predetermine the number of individuals n_i from each front that can be inserted into the population $\mathbf{P}$.

8. Application to optimization of system design

8.1. Description of the problem

The models presented in Sections 4–6 are applied in this and the next section to optimization of SIS. In this section, the application case is for optimization of system design. The optimization is made for three objectives: PFD_{avg}, STR and LCC. The application case is the protective function against high pressure and temperature of a chemical reactor shown in Fig. 3. The system is composed of four subsystems: Temperature transmitter (TT), pressure transmitter (PT), logic solver (LS) and final control element (FC). Upon detection of either high temperature or pressure the safety system cuts the reactor supply off in order to prevent a runaway reaction. The subsystems are MooN redundant (except the FC that is only parallel). It is assumed that in every case the voters are perfect. It is also assumed that the PT and TT must be both fully functional for the safety system to be working. Therefore, the four subsystems are to be modeled in a logical series structure. The data of the problem is shown in Table 6. The case study seeks to explore whether introducing voting architectures as an alternative to simple parallel redundancies in the optimization of design would enable the achievement of better trade-offs between the two dependability attributes (PFD_{avg} and STR), and therefore to attain a better LCC. This can be achieved optimizing the N and M indices of the MooN voting system. Two optimization cases are run in order to contrast their results.

- Case 1. Optimization of system design with option of MooN architectures. All subsystems, excepting the FC, can be subject to voting. The decision variables are: the type of components for the subsystem (three choices per subsystem, distinguished and type 1, 2, 3, excepting for FC that are only two), the number of components N (up to 5), and the required number of voting components M needed to initiate the safety function ($M \leq N$) (for PT, TT and LS). The search space has approximately 4.22×10^6 potential solutions. The vector of decision variables is therefore:

$$\mathbf{x} = \{\text{Type}_{PT}, N_{PT}, M_{PT}, \text{Type}_{TT}, N_{TT}, M_{TT}, \text{Type}_{LS}, N_{LS}, M_{LS}, \text{Type}_{FC}, N_{FC}\} \quad (34)$$

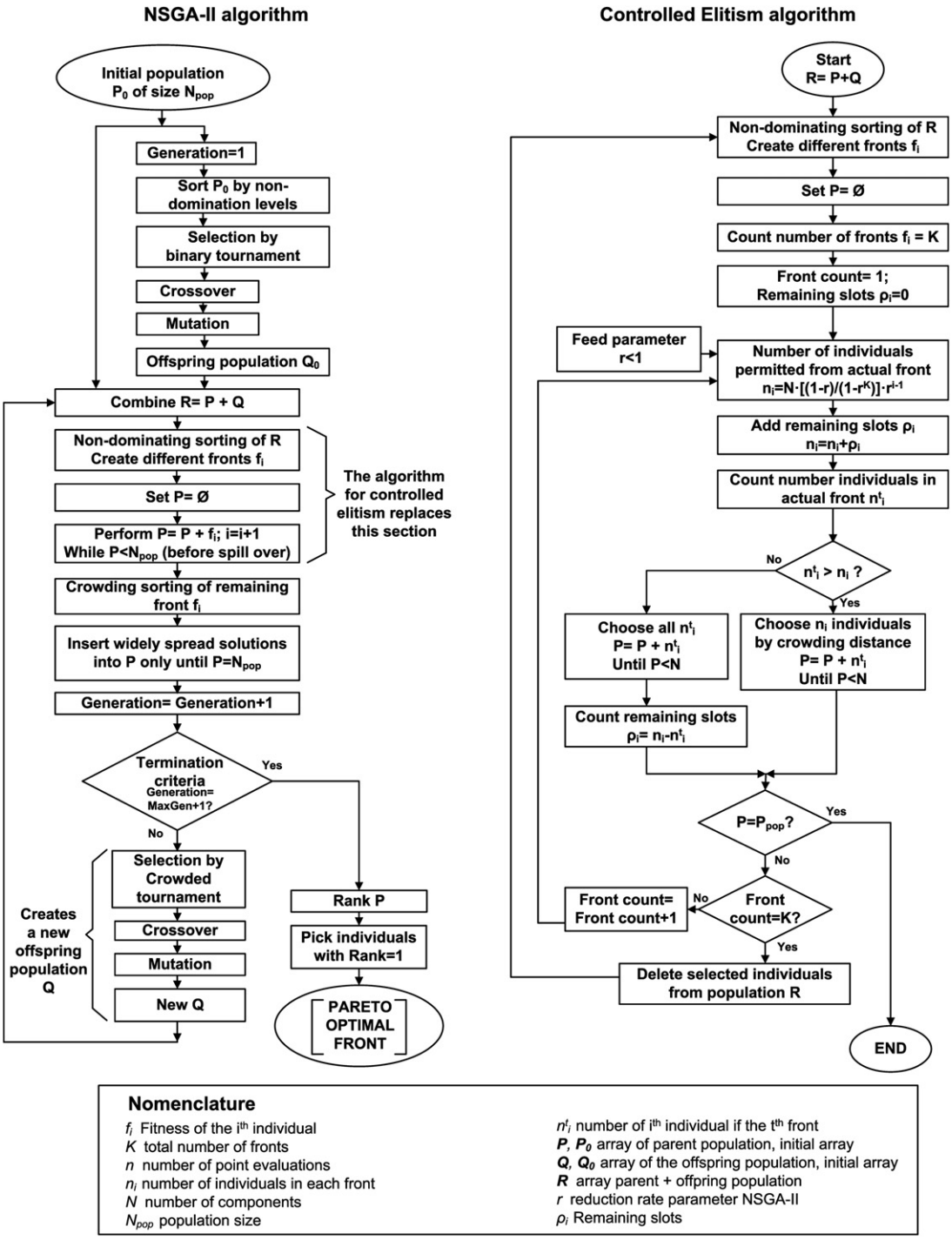


Fig. 2. NSGA-II algorithm and the added controlled elitism algorithm.

- Case 2. Optimization of system design with only parallel architectures. The same as above, but all subsystems are constrained to $M=1$.

8.2. Implementation of the solution

Modeling of the dependability objectives is made with the models developed in Sections 4 and 5. The fault tree for PFD(t) at subsystem level is constructed based on the guidance given by Table 4. The fault tree at system level includes house events that select amongst different architectures for each subsystem, as it is shown in Fig. 4. The tree only exemplifies the PT subsystem, but the other subsystems are developed in the same fashion. The quantification of PFD(t) at subsystem level is based on the model given by Fig. 1(a) and (b). Observe that the PFD(t) model is used to solve complete MooN architectures and they applied to the fault tree as a basic event. The failure rates the model are determined by Eqs. ((6)–(11)). The modification factors C_{MooN} for each architecture can be determined based on Table 2, and their actual values taken from Table 3. The total PFD(t) of the system (Eq. (5)) is evaluated every hour for the entire system operating life, and

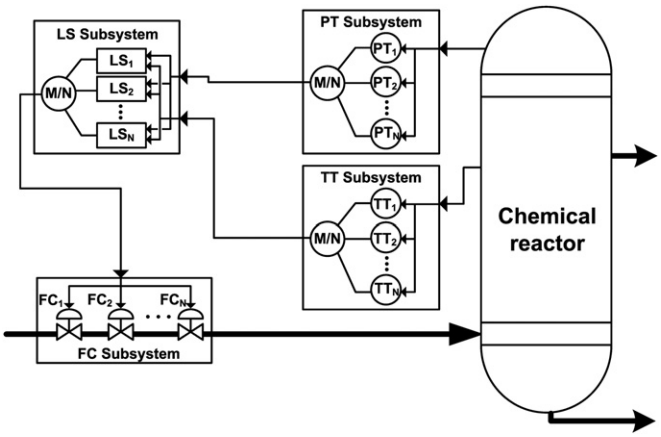


Fig. 3. Chemical reactor protection system with MooN subsystems.

then these point values are averaged (Eq. (35)).

$$\text{PFDavg} \approx \frac{\sum_{i=0}^{T} \text{PFD}_{\text{TOT}}(t_i)}{n} \tag{35}$$

The fault tree for STR follows the same philosophy of Fig. 4 regarding the application of house events, using the adequate selection of logic gates. At subsystem level the fault tree is constructed and quantified following Table 5 (the values for modification factors are taken from Table 3). Remember that, according to Eq. (12), the total STR of each subsystem is composed by the sum of spurious trip cause by internal failures and test-induced STR. This must be considered in the construction of the fault tree at subsystem level. The STR_{λ} contributions are reconfigured during testing (also determined by Table 5), and summed as indicated by Eq. (14). Failure rates of basic events can be determined applying Eqs. (17)–(21), and the test-induced share by Eq. (13).

Modeling of LCC is made using the model presented in Eqs. (22)–(33). The optimization is executed using the Genetic Algorithm NSGA-II with controlled elitism. The algorithm and its implementation are as detailed in Section 7.

8.3. Discussion of results

The Pareto-optimal sets obtained from both cases are compared in Fig. 5. The plots PFD_{avg} vs STR and PFD_{avg} vs LCC clearly show that the solutions of both fronts with the same level of PFD_{avg} have lower LCC and lower STR in the optimal set of Case 1, where the option to choose MooN architectures was available. For the graph STR vs LCC this advantage is not that clear. However, if one solution of Case 1 is picked, it can be seen that for the same LCC a solution in Case 2 would have a higher STR. In order to make a more rigorous verification of the advantage of Case 1 over Case 2, the solutions belonging to both sets were put together and re-ranked to find only the non-dominated individuals. It was concluded that, in general, the optimal set with MooN architectures dominated the only-parallel one.

Another important observation from Fig. 5 is that the PFD_{avg} is not always in conflict with LCC in the Pareto-optimal front. There is a region where PFD_{avg} and LCC hold a harmonious relationship ($\text{PFD}_{\text{avg}} < 2.2 \times 10^{-3}$), and another where they are in conflict ($\text{PFD}_{\text{avg}} > 2.2 \times 10^{-3}$). This is a result of the balance of costs achieved with the changing design: procurement vs operation costs and safe vs dangerous risk costs. On the other hand, STR is also in a dual relationship with LCC in the front, which change around STR $\sim$0.087/year. Finally, it can be seen that PFD_{avg} and STR are consistently in conflict. Another feature to observe is that a region of high saturation (PFD_{avg} vs LCC) on the left side of the graph is not present as in the cases of previous works [16–18]. A high saturation point appears only if the PFD_{avg} scale is changed to linear.

Observe now Fig. 6, in which the full results are shown for both cases. In this figure, the particular combination of decision variables and resulting objectives are shown for each solution of the optimal set. It is notable that the optimizer consistently selects the components with better specifications despite their higher acquisition cost (mostly Type 1). These components have overall lower failure rates and mostly higher diagnostic coverage. This means that they have a reduced impact in both PFD_{avg} and STR (i.e., lower values) in comparison with the other types of components available. This in turn impacts positively on the overall LCC. Therefore, their benefits in terms of reduced operational costs and risk costs, by both loss of production and loss of safety (i.e., catastrophic hazards) outweigh the expenditure caused by their high initial acquisition cost.

Comparing the solutions from both cases, it is clear that optimization with MooN offers many more options to the optimizer. In the only-parallel case, improvements in PFD_{avg} are gained by adjusting the FC subsystem, then the PT. In these two subsystems lie most of the trade-offs. Once no more improvements are possible changes in the TT and LS subsystems are explored. It can be seen in the optimization with MooN that the trends of the variables of the four objectives are much less flat than the only-parallel case, meaning they change more dynamically to achieve optimal trade-offs.

In Case 1, the FC subsystem (that has only parallel redundancy) dominates the changes in PFD_{avg} and LCC. This suggests that the improvements in PFD_{avg} are largely acquired by increasing the parallel redundancy level of the FCs. This, however, also increases the STR.

It is important to notice how, from left to right, decreasing the PFD_{avg} requires an increment in the number of components in the subsystem N and a reduction in the required number of voting components M needed to initiate the safety function. This is easy to spot in the variables of the PT and TT subsystems. These changes, however, affect the STR in the opposite sense. Therefore, the option of being able to choose between different levels of required number of voting components M provides a better balance of PFD_{avg} against STR. Observe both ends of the PT and TT subsystems. At the far left, with the lowest STR, the voting is NooN. On the contrary, at the far right, with the lowest PFD_{avg}, the voting is 1ooN.

Clearly this trading between the variables N and M also allows balancing of the total LCC associated with the system. Observe that the top graph showing the objectives reiterates the trend in which the PFD_{avg} and STR have a dual relationship with the LCC in the different halves of the graph, the transition of which is marked by solution 61 which has the lowest LCC. The likely explanation is that the risk cost associated to the STR becomes the dominant factor of the LCC after solution 61. Observe the set of solutions 1–61. Trade-offs are both in N and M values of the three MooN subsystems (PT, TT and LS). Also the number of solutions with $M > 1$ (i.e., non-parallel subsystems) is very high, being also with M quite close to N in most of the cases. It was seen in Ref. [20] that, with a fixed number of components N, increments in the voting components M increases the PFD_{avg} (see Section 6 of that reference) but reduces the STR. It is thus clear that a low N/M ratio keeps the STR low (> 0.087), which results in a reduction of PFD_{avg} and also reduces the LCC. On the contrary, for solutions 62–127 the number of solutions with $M > 1$ is largely reduced. The trade-offs increasingly depend on the number of components N. Observe that, in general, the values of M are smaller than for solutions 1–61. What is more, the PT subsystem has mostly $M = 1$ (in solutions 62–127), and the number of components in the FC subsystem is increased significantly. This means that many more solutions in that group (62–127) have subsystems which are simple parallel and with a higher number of components. Also the number

Table 6
Dependability and lifecycle cost data.

Subsystem	PT			TT			LS		FC		
Type	1 Smart transmitter	2 Conventional electronic transmitter	3 Switch	1 Smart transmitter	2 Conventional electronic transmitter	3 Switch	1 Safety PLC	2 Standard PLC	1 Air operated[b]	2 Hydraulic operated[b]	3 Motor operated[b]
$\varepsilon^S/\varepsilon^D$ (%)	69.2/31.8	56.0/51.1	10/10	97.9/7.5	86.9/45.5	10/10	100/81.25	45/60	0/25	0/20	0/10
λ^{SD} ($\times 10^{-6}$/hr)	0.265	1.21	0.68	5.05	6.5	0.92	3.46	1.77	0	0	0
λ^{SU} ($\times 10^{-6}$/hr)	0.118[a]	0.95	6.13	0.11[a]	0.98	8.30	0	2.17	3.94	3.17	9.17
λ^{DD} ($\times 10^{-6}$/hr)	0.048	0.97	0.41	0.026	1.57	0.76	0.026	2.89	0.84	1.09	0.79
λ^{DU} ($\times 10^{-6}$/hr)	0.103	0.93	3.70	0.322	1.88	6.84	0.006	1.92	2.51	4.35	7.11
λ^{T} ($\times 10^{-6}$/hr)	0.534	4.06	10.92	5.508	10.93	16.82	3.492	8.75	7.29	8.61	17.07
Type	B	A	A	B	A	A	B	B	A	A	A
SFF (%)	80.7	77.09	66.12	94.15	82.79	59.33	99.83	78.06	65.57	49.48	58.35
$C^{purchase}$ ($)	4844	2306	500	2560	1406	500	3000[c]	2500[c]	6940	6400	6200

Lifecycle cost data:
Design/install/commissioning PLC=10,320 ($).
Maintenance PLC=960 ($/event).
Test PLC=240 ($/event).
Design overall instrumentation=3,060 ($).
Installation/commissioning per instrument=600 ($).
Maintenance per instrument=240 ($/event).
Test per instrument=60 ($/event).
Repair cost per instrument & PLC=60 ($/hour).
Spares per repair=25% component cost.
Loss of production=2000 ($/hour).
Start up cost=1800 ($).
Catastrophic loss=150×10^6 ($).
SIS life=15 (years).
Discount rate=0.05.
Other data:
Cost of rack of safety PLC: 31,000 ($).
Cost of rack of standard PLC: 20,500 ($).
Repair time=8 (hrs).
Shut down time=24 (h).
Test interval=1 (year).
Plant risk without SIS=8.55 ($\times 10^{-3}$/yr).
β=10%[d].

[a] This failure mode does not cause spurious trip, only quantified for SFF.
[b] Includes actuator.
[c] The PLC must be bought with a rack which cost is indicated above.
[d] Apply for all cases, except for β^{SD} of: PT1=5%, TT1=2%, TT2=5%.

of subsystem with $M>1$ have a larger N/M ratio. This situation increments the STR consistently above 0.087, becoming a dominant factor of the LCC. Observe that the LCC practically follows the trend of the STR for solutions 62–127. In conclusion: systems with a higher level of voting (lower N/M ratio) achieve lower values of STR greatly benefiting the LCC. However they have higher values of PFD_{avg}. On the contrary, increasing the number of components and using more parallel architectures decreases the PFD_{avg} (i.e., safer systems) but increases the STR and, to some extent, the LCC.

9. Application to optimization of testing policies

9.1. Description of the problem and solution

This second case study is about application of the models to optimization of testing polices. The theory of testing polices and their optimization has been detailed in Ref. [18]. The reader may want to consult that source for better understanding of the implementation of this case study. A proof testing policy includes the Test Interval (TI, which determines the frequency), and the Test Strategy (TS). These two last items, denominated together TI+TS, can be manipulated to find an optimal solution. Any change in the policy impacts on the three system objectives PFD_{avg}, STR and LCC.

The test strategy can be sequential or staggered test (with varying levels of staggering). The difference of staggering can give uniform and non-uniform strategies. The degree of distribution of the test events of different components along the TI of the staggered test strategy is commanded by a factor K (determined by a multiplication factor P (Eq. (37)) that determines the difference of time between the time of first test of the first component (T_{P1}) and the rest of them (Eq. (36)). Therefore the variables of the TS are the T_{P1} and the factor P. These, together with the TI, compose the three decision variables of the test policy. The T_P per component is thus determined as follows:

$$T_{P1}=T_P,\quad T_{P2}=T_{P1}+K,\quad T_{P3}=T_{P1}+2K,\ldots\quad T_{PN}=T_{P1}+(N-1)K \tag{36}$$

where:

$$K=\mathrm{Round}\left(\frac{TI}{N}\cdot P\right) \tag{37}$$

Please do not confuse the multiplication factor P used in Eq. (37) with the population array **P** used in the NSGA-II algorithm (Fig. 2), since they are completely different things.

The design of the reactor protective system is the same as in Fig. 3, but with fixed number of components N in the subsystems: three PTs, two TTs, four LSs and two FCs. The technology used in the subsystem is as follows: conventional pressure transmitter,

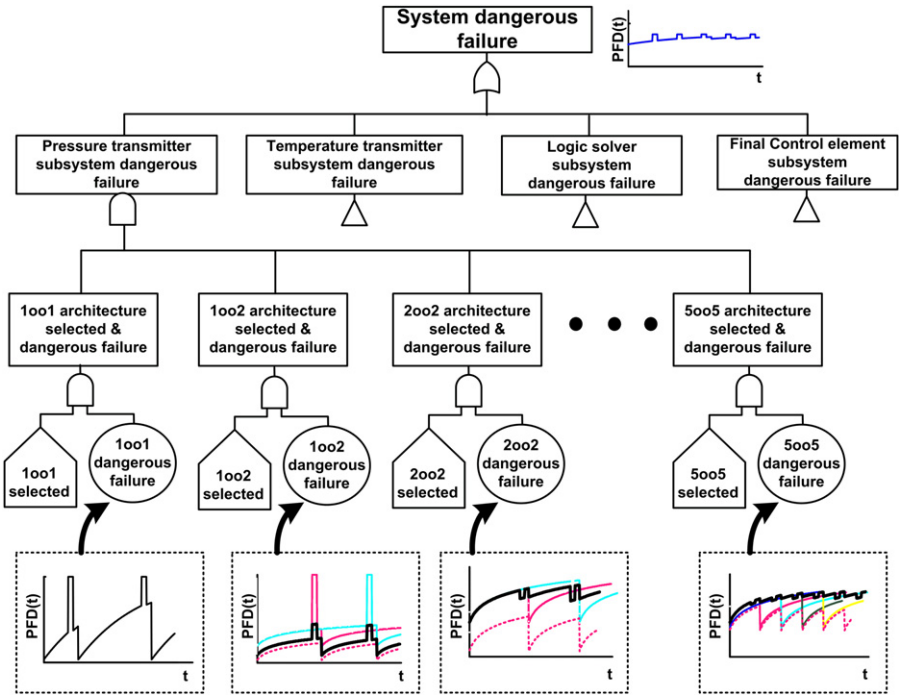


Fig. 4. Fault tree for quantification of PFD with optional MooN architectures.

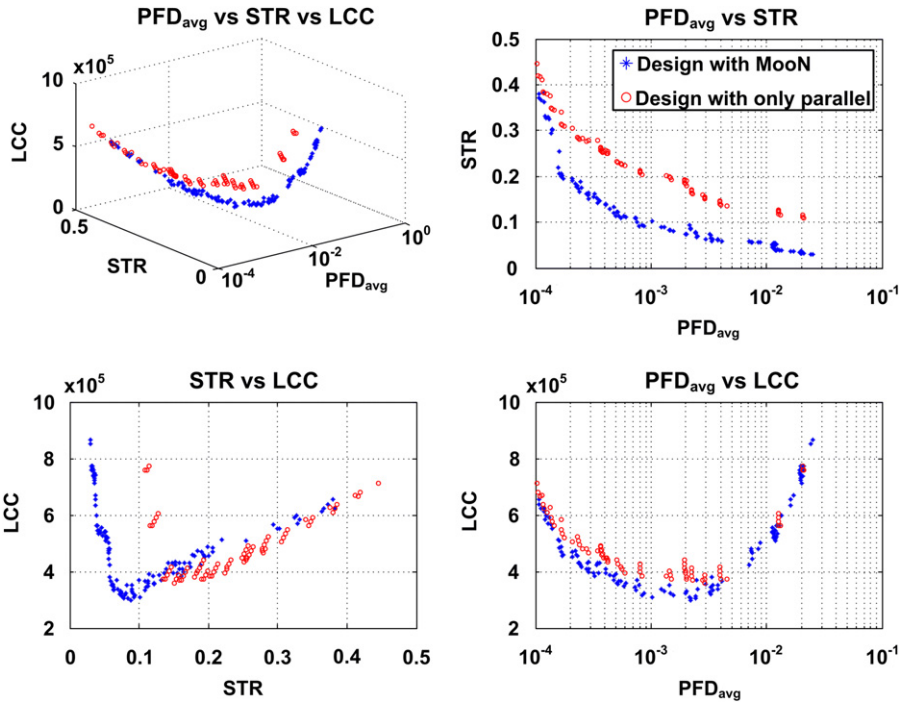


Fig. 5. Comparison of optimal sets of both design optimization cases.

smart temperature transmitter, safety PLC and air operated valves. All data has already been provided in Table 6.

Ref. [18] verified that proof testing has both positive and negative impact on the system dependability and LCC. This case study aims to explore whether the optimization of testing policies where additionally the voting scheme (M) of some MooN subsystems can be chosen could permit the implementation of better testing polices, counteracting the negative impact of testing while having the same benefits in terms of safety and cost. This requires optimization of the testing policy at subsystem level together with the number of required voting components M needed to initiate the safety function.

Two optimization cases are carried out:

- Case 3. Optimization of testing polices with option of MooN architectures. The TI is bounded between 30 and 365 day (730 for the LS), and T_{P1}s between 0 and TI. The multiplication factor P is a number between 0 and 1000 (then multiplied by 1×10^{-3} so that $P<1$). In addition the number of voting components M of the PT, TT and LS subsystem can be chosen

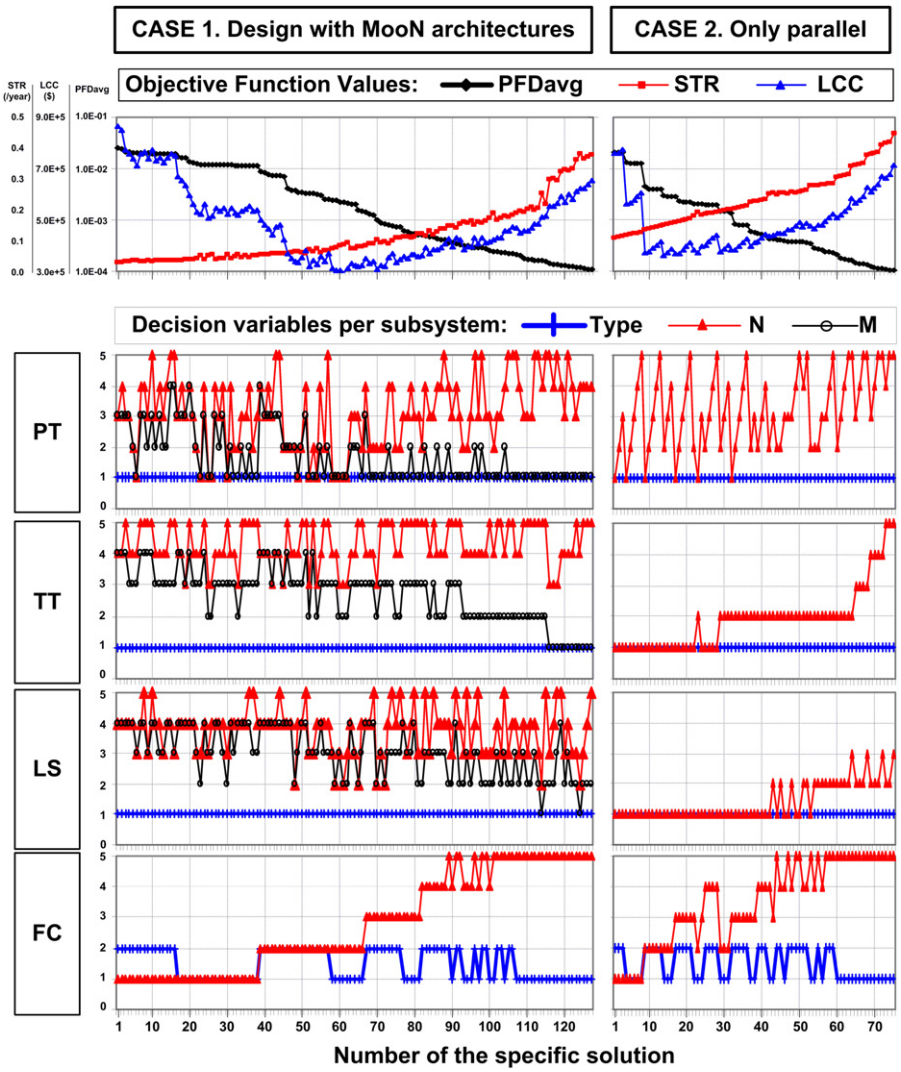


Fig. 6. Results obtained from both design optimization cases.

(subject to $M \leq N$). The total number of potential solutions is 1.68×10^{31}. The decision variables vector is then:

$$\mathbf{x} = \{M_{PT}, TI_{PT}, Tp_{1PT}, P_{PT}, M_{TT}, TI_{TT}, Tp_{1TT}, P_{TT}, M_{LS}, TI_{LS}, Tp_{1LS}, P_{LS}, TI_{FC}, Tp_{1FC}, P_{FC},\} \quad (38)$$

- Case 4. Optimization of testing policies only. The same as above, but all subsystems have just simple parallel redundancy.

The execution of the case study is also made in a similar fashion as for the one in the previous section using the NSGA-II algorithm. For Case 3, house events in the fault trees enable the change in voting components M when selecting amongst different architectures constrained by the number N of each subsystem. Construction of the fault tree for STR follows the same philosophy. Variations in the testing variables (TI, T_{P1} and P) are incorporating by feeding them into the equations for dependability modeling.

9.2. Discussion of results

The Pareto-optimal sets obtained from both optimization cases are compared in Fig. 7. The first notable feature to observe is the domination of the optimal set of Case 3 over the optimal set of Case 4. It is clear from observing the graphs PFD_{avg} vs STR and PFD_{avg} vs LCC that, for the same levels of PFD_{avg}, the costs in terms of STR and LCC are lower for Case 3 (optimization of testing together with voting M). It is also shown, in the STR vs LCC graph, that solutions with the same level of LCC have lower STR in Case 3. An exercise in which the solutions of both optimal sets were put together and re-ranked to obtain the only optimal ones showed that, from a set of 149 solutions, only three of them belonged to the optimal set of Case 4. Since these belong also to the search space of Case 3, it is believed that new runs of the algorithm could find them. Therefore, it can be concluded that the optimization of testing polices together with the level of voting M gives results that dominate those from optimization without M.

Analyzing the relationships between the objectives in the optimal set of Case 3, it is shown that the PFD_{avg} has a dual relationship with the LCC. There is a region of harmonious relationship ($PFD_{avg} > 1.09 \times 10^{-3}$), and another region of conflicting relationship ($PFD_{avg} < 1.09 \times 10^{-3}$). The PFD_{avg} and STR (as with the cases analyzed in previous chapters) have a consistently conflictive relationship. The STR and LCC tend also towards conflict.

Fig. 8 shows the complete results of both optimization cases, including the values of the objectives and decision variables of each solution belonging to the Pareto-optimal sets. Observe first the results given by Case 4. The PFD_{avg} is reduced by primarily lowering the TI of the FC subsystem (the time to first test T_{p1} is

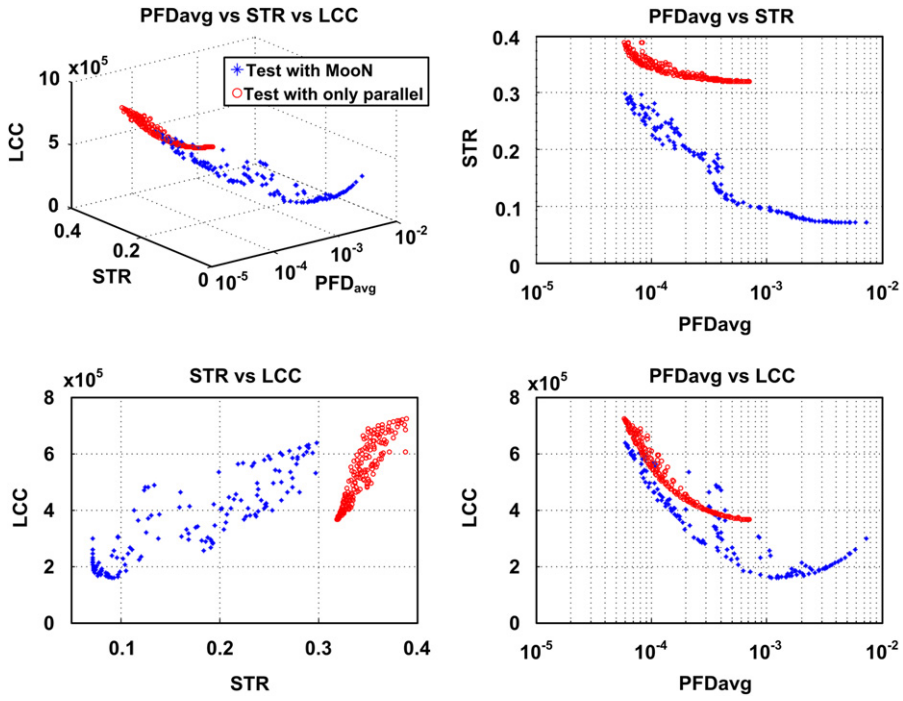


Fig. 7. Comparison of optimal sets of both testing optimization cases.

already low). Notice that the trend is to reduce the TI at the lowest possible level (30 day). Once this is done, the TI of the PT and TT subsystems are also lowered to achieve further reduction in the PFD_{avg}. The relationship of the PFD_{avg} vs the LCC and STR is always conflictive in this optimization case. Therefore, the action described consistently raises these two objectives. Recall that the FC subsystem is the one with the lowest level of redundancy (1oo2) and with the less reliable components. Therefore the reduction of PFD_{avg} depends largely on its testing policy. On the contrary, the LS that has high redundancy (1oo4) and highly-reliable components remains with a high TI; i.e., above 400 day (low testing frequency). Also notice that the multiplication factor P (Eq. (34)) of every subsystem, excepting LS, tends towards a high value, i.e., towards full uniform staggering.

Now observe the results from Case 3. In comparison with Case 4, solutions with the same value of PFD_{avg} have lower LCC and STR values. This is notably evident for the STR, where the difference of values can be noticed at a single glance at the upper part of Fig. 8. Note, however, that the TI and the T_{p1} values of the PT, TT and FC subsystems remain at very low levels. Their values are even lower than for the system with only-parallel redundancies (Case 4). This means higher testing frequencies and earlier in the system lifecycle. Nevertheless, in an opposite sense, the multiplication factor P (in Case 3) has lower and more changing values than in Case 4. This means that the distribution of the test events along the test interval has a lower trend to be uniform (full staggering).

Remember that the only difference between cases 3 and 4 lies in the fact that Case 3 permits voting (i.e., M is a decision variable) and Case 4 is limited to parallel redundancies (i.e., $M=1$). Therefore, it is possible to that say introducing voting (permitting M to be varied between 1 and N) causes lower values of P in the optimal set of Case 3. This in turn suggests that trade-offs are made between the level of voting M and the level of test staggering (determined by P) to achieve lower values of PFD_{avg}. This strongly suggests that by trading between the voting level M and the test staggering P (with low TIs), low levels of PFD_{avg} can be achieved with lower values of STR and LCC rather than when varying only the testing parameters (with $M=1$).

This conclusion motivates a deeper analysis. It is clear that introducing voting in the subsystem (i.e., $M>1$) lowers the level of staggering in the optimal set (the variable P). Even in the FC subsystem that has only parallel redundancies the values of P are also lower (than in Case 4). Therefore, the introduction of voting in some subsystems causes lower levels of P in all subsystems. Also notice that even for solutions 124 to 149 (Case 3), where $M=1$ for all subsystems except for LS, the values of P are still lower than in Case 4. Also observe that in Case 3 the same values of PFD_{avg} in both cases correspond to lower STR and LCC figures (than in Case 4). All these remarks lead to the conclusion that the trade-offs between M and P are at the system level.

Observe also that the level of reduction of P is more prominent in the TT and FC subsystems, in that order. This can be explained observing Table 6, where it is seen that the total failure rate per component λ^T is higher in the smart transmitter and the air operated valves than in the conventional pressure transmitter and the safety PLC (the technologies used in this case study). In addition, compare the safe and dangerous failure rates of the smart temperature transmitter against the conventional pressure transmitter. The PT has higher dangerous failure λ^D rates than the TT, but the TT has significantly higher safe failure rates λ^S than the PT. Thus, is can be said that the staggering P is lowered in the TT to decrease the STR (which is affected by higher λ^S).

Observe now in the results given by Case 3 that the test intervals are also lower than in Case 4; i.e., higher testing frequency. This is true for all subsystems, although more notable in the field instruments (PT, TT and FC). This observation can be reinforced by comparing the group of solutions that are between the same band of values of PFD_{avg} in both cases. These are solutions 50–149 in Case 3 against the complete set of Case 4. The TIs in Case 3 solutions are consistently lower. Therefore, it is evident that there is also a trade-off between TI and P. Since proof testing is aimed at dangerous failures, this suggests that once voting is introduced in the form of the variable M, the optimizer chooses to test more frequently the dangerous failures in all subsystems (with lower TIs) in exchange of less staggered testing especially in the subsystems with higher safe failure rates.

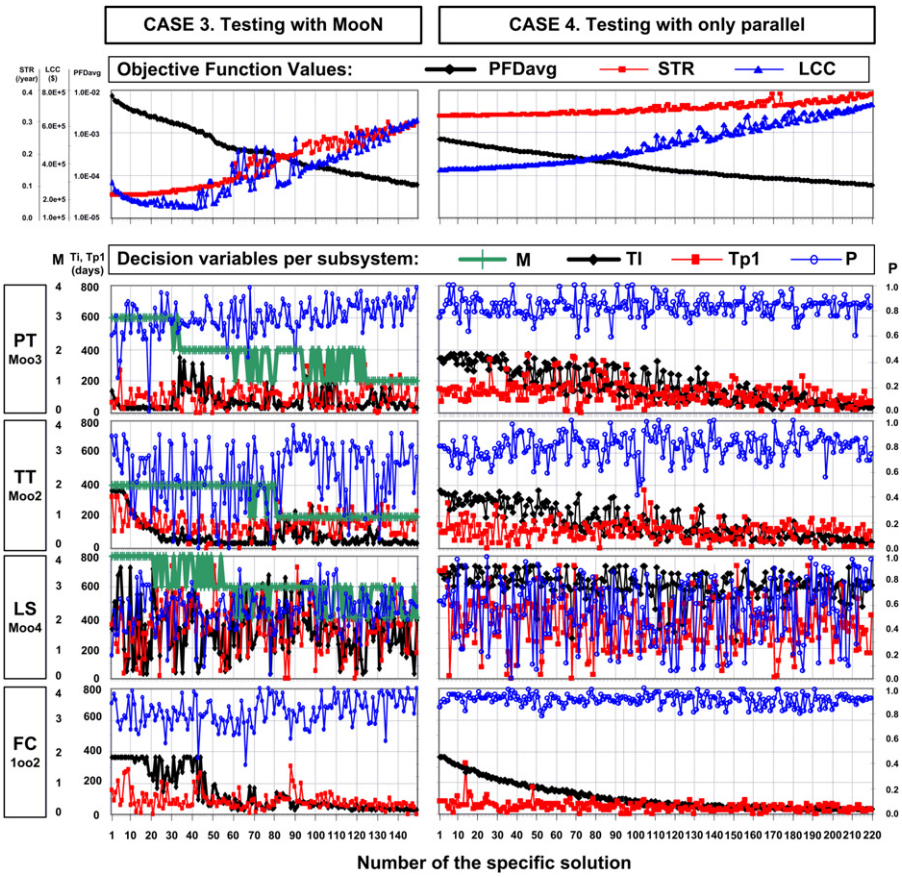


Fig. 8. Results obtained from both testing optimization cases.

Trade-offs between the variables M and TI can also be seen in some specific solutions of Case 3. For instance, observe solutions 94–96 in the PT subsystem. The value of M decreases and TI increases simultaneously. Also in solutions 125–149 of the PT subsystem the variable M becomes equal to 1 at the same time that the values of TI are raised. For the TT subsystem there is a rise in TI values after solution 81, where M becomes equal to 1. Thus, it is clear that reduction of M causes TI to rise.

Considering this analysis, it can be said that introducing voting levels as the variable M in the subsystems enables trade-offs between this and the TI and P variables. These are generally in the form of higher testing frequency in exchange of less staggered test. This trading in turn allows achievement of similar values of PFD_{avg} with lower STR and LCC than with only parallel architectures.

It is customarily believed that full staggering (the uniform distribution of the test events of the N components along the test interval; i.e., TI/N) is the best option. It can be seen in this exercise, however, that reducing the level of staggering can actually bring benefits when combined with the changes in the voting scheme M and the test intervals. Nevertheless, it is important to notice that very low staggering (i.e., low values of P) can affect negatively the LCC, which can be seen in solutions 71, 75 and 90 that present high sudden peaks in LCC. As it was seen above, in Case 4 the reduction in PFD_{avg} is primarily achieved by lowering the TI and T_{p1} of the FC, TP and TT and FC subsystems. In Case 3, this is achieved by reducing in addition the number of voting components M towards simple parallel ($M=1$) of the TP and TT subsystems. It is evident that the addition of the voting level M as a variable in the optimization of testing policies opens the possibility of new interesting trade-offs that enable the achievement of overall better solutions.

The conclusions drawn above are believed to be applicable to low-demand safety systems under similar conditions as those assumed in this paper; especially constant and low failure rates. This is a promising result that can be the ground for further research with the objective of determining whether this could become a generic design lesson for safety systems.

On observations related to the LCC, the graph of objective's values for Case 3 shows that the trend of the LCC is to be first reduced and later incremented (congruent with the dual relationship PFD_{avg} vs LCC described above). This may be explained by the changing influence of the costs caused by the STR over the LCC. It is suggested that by varying M together with the testing policy variables permits lowering of the STR to such a level that it does not considerably affect the LCC (solutions 1–42). It is in this region that the LCC is in harmony with the PFD_{avg}. Subsequently, the STR increases, when reducing the values of the M variables, in such a way that seems to exert considerable influence over the LCC, and even dominates it when all $M=1$ (i.e., all subsystems have parallel redundancy). Thus, the PFD_{avg} and LCC become conflicting objectives (solutions 43–149).

The group of solutions 1–42 (Fig. 8) where changes in the objectives are pretty smooth correspond to the voting level M of PT and TT being held constant. This region is not explored in the optimization with only parallel architectures. Observe that the highest PFD_{avg} value achieved in Case 4 is 7.13×10^{-4}. In Case 3 solutions 1–50 have higher values than this. Thus exploration in the area of SIL 2 ($\text{PFD}_{\text{avg}}>1\times10^{-3}$) is not present in Case 4. This is another interesting feature of the optimization adding M as variable.

10. Concluding remarks

This paper applies the PFD(t) and STR models recently developed by the authors to optimization of design and testing of

safety instrumented systems. The models have been integrated, together with a Lifecycle Cost model, as objective functions into a multi-objective genetic algorithm. The optimization methodology has been applied to two optimization cases. The first case for optimization of system design (redundancy allocation and component selection) with the option of choosing the number of required voting components M needed to initiate the safety function. The second case is about optimization of testing policies with also the addition of the voting level M as a decision variable.

It was seen that the set of solutions given by the optimization cases with the variable M (voting level) dominated the set of solutions of the optimization cases without it. In other words, for the same values of PFD_{avg}, lower levels of STR and LCC were achievable. These results demonstrated that the inclusion of the voting level M as an additional decision variable (in both optimization cases) permits the achievement of better trade-offs than when this is not considered. In addition, this has allowed extension of the search space considerably. For the test optimization case it even pushed the results to a region not explored by the optimization with only parallel architectures (i.e., SIL 2). In the optimization of system design it opens up the possibility of new trade-offs between the number of components N and the voting components M. In general, to increase the number of components M (with a fixed total number N) raises the PFD_{avg} but reduces the STR. This trading permits the achievement of designs with a better balance between these two dependability measures. In addition, this trade-off has a significant, potentially positive, impact on the system LCC. Important trade-offs have also been found in the optimization of testing policies between the voting level M and the level of staggering of the test events (i.e., distribution of the test events along the test interval) of the redundant components. It is mainly this trade-off that enabled the achievement of equivalent levels of PFD_{avg} with better STR and LCC. It can be finally said, to conclude, that the multi-objective optimization of safety systems is therefore highly enhanced by the inclusion of the voting level M as an additional decision variable.

References

[1] Martorell SA, Melia ME, Marti SM, Garcia JS, Soriano-Melchor F, Verdu-Martin G, Serradel V. Análisis del periodo de pruebas optimo en la revisión de especificaciones técnicas de C.N. relativas a STI's. Aplicación a un sistema por dos componentes en paralelo. XIV Annual Meeting of the Spanish Nuclear Society. Marbella, Spain, 1988.

[2] Vaurio JK. The theory and quantification of common cause shock events for redundant standby systems. Reliability Engineering & System Safety 1994;43(3):289–305.

[3] Vaurio JK. Optimization of test and maintenance intervals based on risk and cost. Reliability Engineering & System Safety 1995;49(1):23–6.

[4] Martorell S, Serradell V, Samanta PK. Improving allowed outage time and surveillance test interval requirements: a study of their interactions using probabilistic methods. Reliability Engineering & System Safety 1995;47(2): 119–129.

[5] Cepin M, Mavko B. Probabilistic safety assessment improves surveillance requirements in technical specifications. Reliability Engineering & System Safety 1997;56(1):69–77.

[6] Cepin M. Optimization of safety equipment outages improves safety. Reliability Engineering & System Safety 2002;77(1):71–80.

[7] Coit DW, Smith A. Reliability optimization of series-parallel systems using a genetic algorithm. IEEE Transactions on Reliability 1996;45(2):254–60.

[8] Coit DW, Smith A. Solving the redundancy allocation problem using a combined neural network/genetic algorithm approach. Computers and Operations Research 1996;23(6):515–26.

[9] Pattison RL, Andrews JD. Genetic algorithms in optimal safety system design. Proceedings of the IMechE Part E: Journal of Process Mechanical Engineering 1999;213(E3):187–97.

[10] Andrews JD, Bartlett LM. Genetic algorithm optimisation of a firewater deluge system. Quality & Reliability Engineering International 2003;19(1): 39–52.

[11] Riauke J, Bartlet L. An offshore safety system optimization using an SPEA2-based approach. Proceedings of the IMechE Partt O: Journal of Risk and Reliability 2008;222(O3):271–82.

[12] Cantoni M, Marseguerra M, Zio E. Genetic algorithms and Monte Carlo simulation for optimal plant design. Rel Eng Syst Saf 2000;68(1):29–38.

[13] Marseguerra M, Zio E, Podofillini LA. Multiobjective Genetic Algorithm Approach to the Optimization of the Technical Specifications of a Nuclear Safety System. Reliability Engineering & System Safety 2004;84(1):87–99.

[14] Mishra A, Patwardhan A, Verma AK. Safety management in NPPs using an evolutionary algorithm technique. Nuclear Engineering and Design 2007; 237(12–13):1145–451.

[15] Tavakkoli-Moghaddam R, Safari J, Sassani F. Reliability optimization of series-parallel systems with a choice of redundancy strategies using a genetic algorithm. Reliability Engineering & System Safety 2008;93(4):550–6.

[16] Torres-Echeverria AC, Thompson HA. Multi-objective genetic algorithm for optimization of system safety and reliability based on IEC 61508 requirements: a practical approach. Proceedings of the IMechE Part O: Journal of Risk and Reliability 2007;221(O3):193–205.

[17] Torres-Echeverria AC, Martorell S, Thompson HA. Design optimization of a safety instrumented system based on RAMS+C addressing IEC 61508 requirements and diverse redundancy. Reliability Engineering & System Safety 2009;94(2):162–79.

[18] Torres-Echeverria AC, Martorell S, Thompson HA. Modelling and optimization of proof testing policies for safety instrumented systems. Reliability Engineering & System Safety 2009;94(4):838–54.

[19] Kancev D, Gjorgiev B, Cepin M. Optimization of test interval for ageing equipment: a multiobjective genetic algorithm approach. Journal of Loss Prevention in the Process Industries 2011;24(4):397–404.

[20] Torres-Echeverria AC, Martorell S, Thompson HA. Modelling safety instrumented systems with MooN voting architectures addressing system reconfigurations for testing. Reliability Engineering & System Safety 2011;96(5): 545–563.

[21] Hauge S, Hokstad P, Langseth H, Oien K. Reliability Prediction Method for Safety Instrumented Systems. PDS Method Handbook.2006 edn. Norway: SINTEF; 2006.

[22] International Electrotechnical Commission. IEC 61508 Ed. 2.0 Functional Safety of Electrical/Electronic/Programmable Electronic Safety-Related Systems. Parts 1-7. Switzerland, 2010.

[23] Mosleh A, Fleming KN, Parry GW, Paula HM, Worledge DH, Rasmuson DM. Procedures for treating common cause failures in safety and reliability studies. Vol. 1: Procedural Framework and Examples. Vol. 2: Analytical Background and Techniques. NUREG/CR-4780 (EPRI NP-5613). USA: Nuclear Regulatory Commission; 1988.

[24] Lu L, Jiang J. Analysis of on-line maintenance strategies for k-out-of-n standby safety systems. Reliability Engineering & System Safety 2007;92(2): 144–155.

[25] Deb K. Multiobjective Optimization Using Evoluationary Algorithms. Chichester, UK: Wiley; 2001.

[26] Marseguerra M, Zio E, Martorell S. Basics of genetic algorithms optimization for RAMS applications. Reliability Engineering & System Safety 2006;91(9): 977–991.

[27] Konak A, Coit DW, Smith AE. Multiobjective optimization using genetic algorithms: a tutorial. Reliability Engineering & System Safety 2006;91(9): 992–1007.

[28] Martorell S, Carlos S, Sanchez A, Serradell V. Constrained optimization of test intervals using a steady-state genetic algorithm. Reliability Engineering & System Safety 2000;67(3):215–32.

[29] Martorell S, Villanueva JF, Carlos S, Nebot Y, Sánchez A, Pitchard JL, Serradell V, RAMS+ C. Informed decision-making with application to multi-objective optimization of technical specifications and maintenance using genetic algorithms. Reliability Engineering & System Safety 2005;87(1):65–75.

[30] Villanueva JF, Sanchez AI, Carlos S, Martorell S. Genetic algorithm-based optimization of testing and maintenance under certain unavailability and cost estimation: a survey of strategies for harmonizing evolution and accuracy. Reliability Engineering & System Safety 2008;93(12):1830–41.

[31] Deb K, Agrawal S, Pratab A, Meyarivan T. A fast elitist non-dominated sorting genetic algorithm for multi-objective optimization: NSGA-II. In: Marc Schoenauer, et al., Eds. Proceedings of the Parallel Problem Solving from Nature 6th Conference. Paris, France, 2000. Lecture Notes in Computer Science no. 197. London: Springer; 2000. pp. 849–858.

[32] Deb K, Goel T. Controlled elitism non-dominated sorting genetic algorithm for better convergence. In: Zitzler E, Ed. Proceedings of the First International Conference on Evolutionary Multi-Criterion Optimization 2001. Zurich, Germany, 2001. Lecture Notes in Computer Science no. 1993. London: Springer; 2001. pp. 67–81.